

6

Understanding the Strategy, Abstract Factory, and Singleton Design Patterns

This chapter explores object creation using a few classic, simple, and yet powerful design patterns from the **Gang of Four (GoF)**. These patterns allow developers to encapsulate behaviors, centralize object creation, add flexibility to their design, or control object lifetime. Moreover, they will most likely be used in all software you build directly or indirectly in the future.

GoF

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides are the authors of *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), and are also known as the **GoF**. In that book, they introduce 23 design patterns, some of which we look at in this book.

Why are they that important? Because they are the building blocks of robust object composition and they help to create flexibility and reliability. Moreover, in *Chapter 7, Deep Dive into Dependency Injection*, we leverage dependency injection to make those patterns even more powerful!

But first things first. The following topics will be covered in this chapter:

- The Strategy design pattern
- The Abstract Factory design pattern
- The Singleton design pattern

The Strategy design pattern

The Strategy pattern is a behavioral design pattern that allows us to change object behaviors at runtime. We can also use this pattern to compose complex object trees and rely on it to follow the **Open/Closed Principle (OCP)** without much effort.

As a follow-up on that last point, the Strategy pattern plays a significant role in the *composition over inheritance* way of thinking. In this chapter, we focus on the behavioral part of the Strategy pattern. In the next chapter, we cover how to use the Strategy pattern to compose systems dynamically.

Goal

The Strategy pattern's goal is to extract an algorithm (strategy) from the host class needing it (the context). That allows the consumer to decide on the strategy (algorithm) to use at runtime.

For example, we could design a system that fetches data from two different types of databases. Then we could apply the same logic over that data and use the same user interface to display it. To achieve this, using the Strategy pattern, we could create two strategies, one named `FetchDataFromSql` and the other `FetchDataFromCosmosDb`. Then we

could plug the strategy that we need at runtime in the `Context` class. That way, when the consumer calls the `Context`, the `Context` does not need to know where the data comes from, how it is fetched, or what strategy is in use; it only gets what it needs to work, delegating the fetching responsibility to an abstracted strategy.

Design

Before any further explanation, let's take a look at the following class diagram:

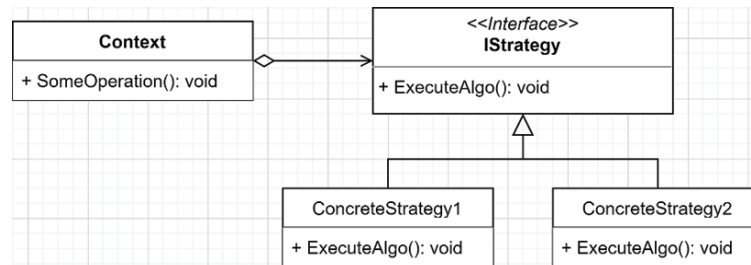


Figure 6.1: Strategy pattern class diagram

The building blocks of the Strategy pattern go as follows:

- `Context` is a class that delegates one or more operations to an `IStrategy` implementation.
- `IStrategy` is an interface defining the strategies.
- `ConcreteStrategy1` and `ConcreteStrategy2` represent one or more different concrete implementations of the `IStrategy` interface.

In the following diagram, we explore what happens at runtime. The actor represents any code consuming the `Context` object.

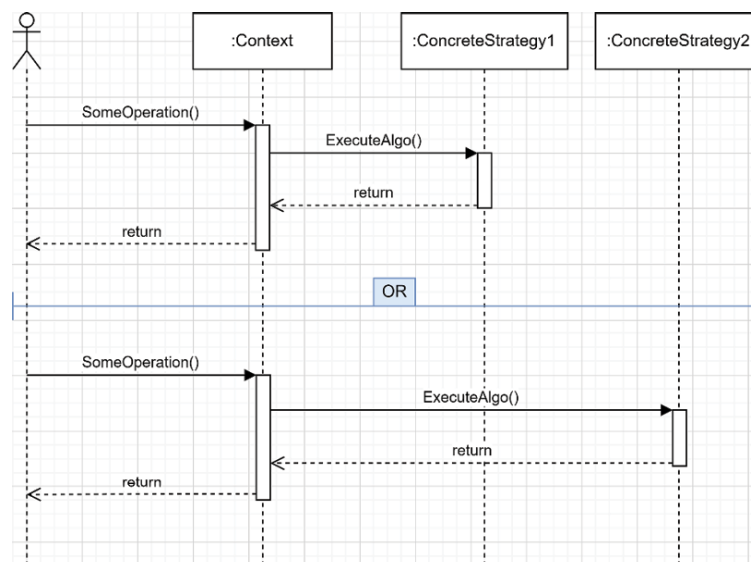


Figure 6.2: Strategy pattern sequence diagram

When the consumer calls the `Context.SomeOperation()` method, it does not know which implementation is executed, which is an essential part of this pattern. `Context` should not be aware of the strategy being used either. It should execute it through the interface without any knowledge of the implementation past that point. That is the strength of the

Strategy pattern: it abstracts the implementation away from both the Context and the consumer. Because of that, we can change the strategy during either the object creation or at runtime without the object knowing, changing its behavior on the fly.

Note

We could even generalize that last sentence and extend it to the use of any interface. Using an interface removes the ties between the consumer and the implementation by relying on the abstraction instead.

Project – Strategy

Context: We want to sort a collection using different strategies. Initially, we want to support sorting the elements of a list in ascending or descending order.

To achieve this, we need to implement the following building blocks:

- The Context is the `SortableCollection` class.
- The Strategy is the `ISortStrategy` interface.
- The concrete strategies are:
 1. `SortAscendingStrategy`
 2. `SortDescendingStrategy`

The consumer is a small program that allows the user to choose a strategy, sort the collection, and display the items. Let's start with the `ISortStrategy` interface:

```
public interface ISortStrategy
{
    IEnumerable<string> Sort(IEnumerable<string> input);
}
```

That interface contains only one method that expects a collection of strings as input, and that returns an ordered collection of strings. Now let's inspect the two implementations:

```
public class SortAscendingStrategy : ISortStrategy
{
    public IEnumerable<string> Sort(IEnumerable<string> input)
        => input.OrderBy(x => x);
}
public class SortDescendingStrategy : ISortStrategy
{
    public IEnumerable<string> Sort(IEnumerable<string> input)
        => input.OrderByDescending(x => x);
}
```

Both implementations are super simple as well, using **Language Integrated Query (LINQ)** to sort the input and return the result directly. Both implementations use **expression-bodied methods**, which we talked about in *Chapter 4, The MVC Pattern Using Razor*.

Tip

When using expression-bodied methods, please ensure that you do not make the method harder to read for your colleagues by creating very complex one-liners. Writing multiple lines often makes the code clearer except in the case of tiny methods like in the preceding example.

The next building block to inspect is the `SortableCollection` class. It is not a collection in itself (it does not implement `IEnumerable` or other collection interfaces), but it is composed of multiple string items (the `Items` property) and can sort them using an `ISortStrategy`, like this:

```
public sealed class SortableCollection
{
    public ISortStrategy? SortStrategy { get; set; }
    public IEnumerable<string> Items { get; private set; }
    public SortableCollection(IEnumerable<string> items)
    {
        Items = items;
    }
    public void Sort()
    {
        if (SortStrategy == null)
        {
            throw new NullReferenceException("Sort strategy not found.");
        }
        Items = SortStrategy.Sort(Items);
    }
}
```

This class is the most complex one so far, so let's take a more in-depth look:

- The `SortStrategy` property holds a reference to an `ISortStrategy` implementation (that can be `null`).
- The `Items` property holds a reference to the collection of strings contained in the `SortableCollection` class.
- We set the initial `IEnumerable<string>` when creating an instance of `SortableCollection`, through its constructor.
- The `Sort` method uses the current `SortStrategy` property to sort the `Items`. When there is no strategy set, it throws a `NullReferenceException`.

With that code, we can see the Strategy pattern in action. The `SortStrategy` property represents the current algorithm, respecting an `ISortStrategy` contract, which is updatable at runtime. The `SortableCollection.Sort()` method delegates the work to that `ISortStrategy` implementation (the concrete strategy). Therefore, changing the value of the `SortStrategy` property leads to a change of behavior of the `Sort()` method, making this pattern very powerful yet simple.

Let's experiment with this by looking at `MyConsumerApp`, a console application that uses the previous code:

```
public class Program
{
    private static readonly SortableCollection _data = new SortableCollection(new[] {
```

The `_data` instance represents the **context**, our sortable collection of items. Next, an empty `Main` method:

```
public static void Main(string[] args) { /*...*/ }
```

To keep it focused on the pattern, I took away the console logic from the book, which is irrelevant for now, but the code is available in the GitHub repository.

```
private static string SetSortAsc()
{
    _data.SortStrategy = new SortAscendingStrategy();
    return "The sort strategy is now Ascending";
}
```

The preceding method sets the **strategy** to a new instance of `SortAscendingStrategy`.

```
private static string SetSortDesc()
{
    _data.SortStrategy = new SortDescendingStrategy();
    return "The sort strategy is now Descending";
}
```

The preceding method sets the **strategy** to a new instance of `SortDescendingStrategy`.

```
private static string SortData()
{
    try
    {
        _data.Sort();
        return "Data sorted";
    }
    catch (NullReferenceException ex)
    {
        return ex.Message;
    }
}
```

The `SortData` method calls the `Sort()` method, which delegates the call to an optional `ISortStrategy` implementation.

```
private static string PrintCollection()
{
    var sb = new StringBuilder();
    foreach (var item in _data.Items)
    {
        sb.AppendLine(item);
    }
    return sb.ToString();
}
```

This last method displays the collection in the console to visually validate the correctness of the code.

When we run the program, the following menu appears:

```
C:\WINDOWS\system32\cmd.exe

Options:
1: Display the items
2: Sort the collection
3: Select the sort ascending strategy
4: Select the sort descending strategy
0: Exit
-----
Please make a selection:
```

Figure 6.3: Output showing the Options menu

When a user selects an option, the program calls the appropriate method, as described earlier.

When executing the program, if you display the items (1), they appear in their initial order. If you assign a strategy (3 or 4), sort the collection (2), then display the list again, the order will have changed and will now be different, based on the selected algorithm.

Let's analyze the sequence of events when you select the following options:

1. Select the sort ascending strategy (3).
2. Sort the collection (2).

Next is a sequence diagram that represents this:

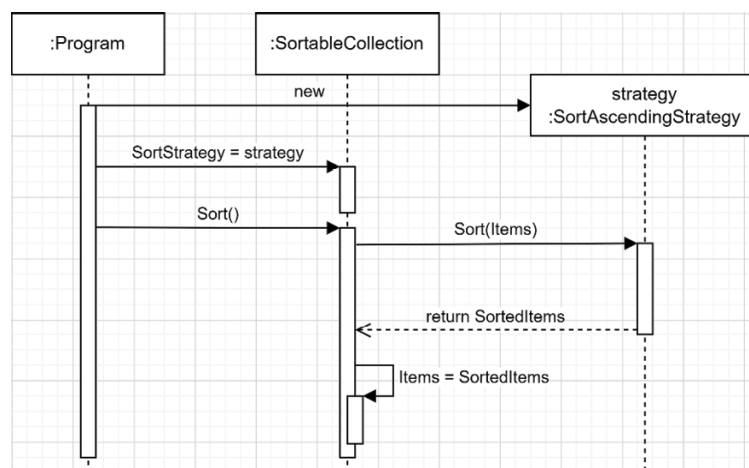


Figure 6.4 : Sequence diagram sorting the items using the “sort ascending” strategy (options 3 then 2)

The preceding diagram shows the `Program` creating a strategy and assigning it to `SortableCollection`. Then, when the `Program` calls the `Sort()` method, the `SortableCollection` instance delegates the sorting computation to the underlying algorithm implemented by the `SortAscendingStrategy` class, a.k.a. the **strategy**.

From the pattern standpoint, the `SortableCollection` class, a.k.a. the **context**, is responsible for keeping a hold on the current **strategy** and for using it.

Conclusion

The Strategy design pattern is very effective at delegating responsibilities to other objects, allowing you to delegate the responsibility of an algorithm to other objects while keeping its usage trivial. It also allows having a rich interface (context) with behaviors that can change during the program's execution.

The strategy does not have to be exposed directly; it can also be private to the class, hiding its presence to the outside world (the consumers); we talk more about this in the next chapter. Meanwhile, the Strategy pattern is excellent at helping us follow the **SOLID** principles:

- **S**: It helps to extract responsibilities to external classes and use them, interchangeably, later.
- **O**: It allows extending classes without updating its code by changing the current strategy at runtime.
- **L**: It does not rely on inheritance. Moreover, it plays a large role in the *composition over inheritance principle*, helping us avoid inheritance altogether and, at the same time, the LSP.
- **I**: By creating smaller strategies based on lean and focused interfaces, the Strategy pattern is an excellent enabler for respecting the ISP.
- **D**: The creation of dependencies is moved from the class using the strategy (the context) to the class's consumer. That makes the context depend on abstraction instead of implementation, inverting the flow of control.

C# Features

If you looked at the implementation of the `Main` method (omitted here), you might have noticed that I used a few newer C# features like **default literal expressions**, **switch expressions**, and **discards**. Those are covered in *Appendix A*.

Next, let's explore the Abstract Factory pattern.

The Abstract Factory design pattern

The Abstract Factory design pattern is a creational design pattern from the GoF. We use creational patterns to create other objects, and factories are a very popular way of doing that.

The Strategy pattern is the backbone of dependency injection, enabling the composition of complex object trees, while factories are used to create some of those complex objects that can't be assembled automatically by a dependency injection library. More on that in the next chapter.

Goal

The Abstract Factory pattern is used to abstract the creation of a family of objects. It usually implies the creation of multiple object types within that family. A family is a group of related or dependent objects (classes).

Let's think about creating automotive vehicles. There are multiple types of vehicles, and for each type, there are multiple models. We can use the Abstract Factory pattern to make our life easier for this type of scenario.

Note

There is also the *Factory Method* pattern, which focuses on creating a single type of object instead of a family. We only cover Abstract Factory here, but we use other types of factories later in the book.

Design

With Abstract Factory, the consumer asks for an abstract object and gets one. The factory is an abstraction, and the resulting objects are also abstractions, decoupling the object creation from the consumers.

That allows adding or removing families of objects produced together without impacting the consumers (all actors communicate through abstractions).

In our case, the family (the object set the factory can produce) is composed of a car and a bike, and each factory (family) must produce both of those objects.

If we think about vehicles, we could have the ability to create low- and high-grade models of each vehicle type. Here is a diagram representing how to achieve that using the Abstract Factory pattern:

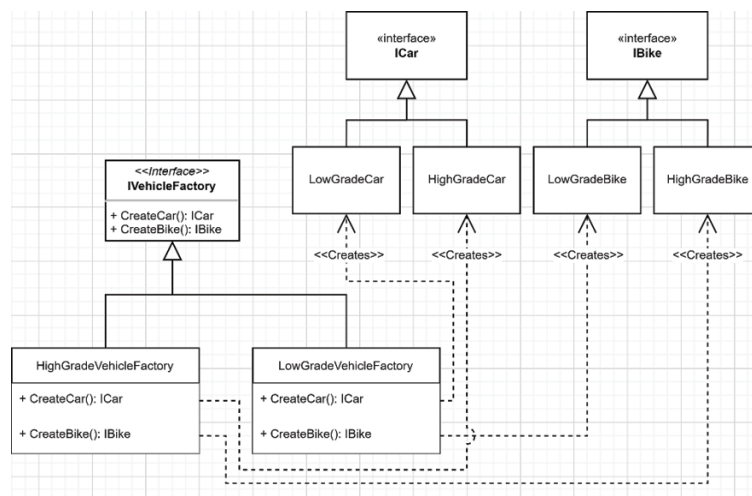


Figure 6.5: Abstract Factory class diagram

In the diagram, we have the following:

- `IVehicleFactory` is an Abstract Factory defining two methods: one that creates cars of type `ICar` and another that creates bikes of type `IBike`.
- `HighGradeVehicleFactory` is an implementation of the Abstract Factory that handles high-grade vehicle model creation. This concrete factory returns instances of type `HighGradeCar` or `HighGradeBike`.
- `LowGradeVehicleFactory` is an implementation of our Abstract Factory that handles low-grade vehicle model creation. This concrete factory returns instances of type `LowGradeCar` or `LowGradeBike`.
- `LowGradeCar` and `HighGradeCar` are two implementations of `ICar`.
- `LowGradeBike` and `HighGradeBike` are two implementations of `IBike`.

Based on that diagram, a consumer uses the `IVehicleFactory` interface and should not be aware of the concrete factory used underneath, abstracting away the vehicle creation process.

Project – AbstractVehicleFactory

Context: We need to support the creation of multiple models of vehicles. We also need to be able to add new models as they become available without impacting the system. To begin with, we only support high-grade and low-grade models and the program only supports the creation of cars and bikes.

For the sake of our demo, the vehicles are just empty classes and interfaces:

```
public interface ICar { }
public interface IBike { }
public class LowGradeCar : ICar { }
public class LowGradeBike : IBike { }
public class HighGradeCar : ICar { }
public class HighGradeBike : IBike { }
```

Let's now look at the part that we want to study—the factories:

```
public interface IVehicleFactory
{
    ICar CreateCar();
    IBike CreateBike();
}
public class LowGradeVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new LowGradeBike();
    public ICar CreateCar() => new LowGradeCar();
}
public class HighGradeVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new HighGradeBike();
    public ICar CreateCar() => new HighGradeCar();
}
```

The factories are simple implementations that describe the pattern well:

- `LowGradeVehicleFactory` creates low-grade models.
- `HighGradeVehicleFactory` creates high-grade models.

The consumer is an xUnit test project. Unit tests are often your first consumers, especially if you are doing **test-driven development (TDD)**.

The `AbstractFactoryBaseTestData` class encapsulates some of our test data classes' utilities and is not relevant to our pattern study. Nevertheless, it can be useful to have all of the code on hand, and it is a very small class; so let's start there:

```
public abstract class AbstractFactoryBaseTestData : IEnumerable<object[]>
{
    private readonly TheoryData<IVehicleFactory, Type> _data = new TheoryData<IVehicleFactory, Type>();
    protected void AddTestData<TConcreteFactory, TExpectedVehicle>()
        where TConcreteFactory : IVehicleFactory, new()
    {

```

```

        _data.Add(new TConcreteFactory(), typeof(TExpectedVehicle));
    }
    public IEnumerable<object[]> GetEnumerator() => _data.GetEnumerator();
    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

```

That class is an `IEnumerable<object[]>` with a private collection of `TheoryData<T1, T2>`, and an `AddTestData<TConcreteFactory, TExpectedVehicle>()` method that is used by other classes, to feed our theories.

The data inheriting from the `AbstractFactoryBaseTestData` class that we are going to feed to our theories looks like this:

```

public class AbstractFactoryTestCars : AbstractFactoryBaseTestData
{
    public AbstractFactoryTestCars()
    {
        AddTestData<LowGradeVehicleFactory, LowGradeCar>();
        AddTestData<HighGradeVehicleFactory, HighGradeCar>();
    }
}
public class AbstractFactoryTestBikes : AbstractFactoryBaseTestData
{
    public AbstractFactoryTestBikes()
    {
        AddTestData<LowGradeVehicleFactory, LowGradeBike>();
        AddTestData<HighGradeVehicleFactory, HighGradeBike>();
    }
}

```

With the implementation details abstracted, the preceding code is straightforward. If we take a closer look at the `AbstractFactoryTestCars` class, it creates two sets of test data:

- A `LowGradeVehicleFactory` that should create a `LowGradeCar` instance.
- A `HighGradeVehicleFactory` that should create a `HighGradeCar` instance.

The same goes for the `AbstractFactoryTestBikes` data:

- A `LowGradeVehicleFactory` that should create a `LowGradeBike` instance.
- A `HighGradeVehicleFactory` that should create a `HighGradeBike` instance.

Now, let's look at the test class and theories using that test data:

```

public class AbstractFactoryTest
{
    [Theory]
    [ClassData(typeof(AbstractFactoryTestCars))]
    public void Should_create_a_Car_of_the_specified_type(IVehicleFactory vehicleFactory,
        ICar expectedCarType)
    {
        // Act
        ICar result = vehicleFactory.CreateCar();
        // Assert
        Assert.IsType(expectedCarType, result);
    }
}

```

```

[Theory]
[ClassData(typeof(AbstractFactoryTestBikes))]
public void Should_create_a_Bike_of_the_specified_type(IVehicleFactory vehicleFac
{
    // Act
    IBike result = vehicleFactory.CreateBike();
    // Assert
    Assert.IsType(expectedBikeType, result);
}
}

```

In the preceding code, we have two theories that each use the data contained in the class, defined by the `[ClassData(...)]` attribute (see the highlighted code). That data is used by the test runner to populate the value of the test method's parameters. So the test runner executes a test once per set of data. In this case, each method runs twice.

The execution of each test method goes as follows:

- We use the Abstract Factory `IVehicleFactory vehicleFactory` to create an `ICar` or an `IBike` instance.
- We test that instance against the expected concrete type to ensure it is the right type; that type is specified by `Type expectedCarType` or `Type expectedBikeType`, depending on the test method.

Note

I used `ICar` and `IBike` to type the variables instead of `var`, to make the type of the `result` variable clearer. In another context, I would have used `var` instead.

We now have four tests; two bike tests

(`Vehicles.AbstractFactoryTest.Should_create_a_Bike_of_the_specified_type`) executed with the following arguments:

```

(vehicleFactory: HighGradeVehicleFactory { }, expectedBikeType: typeof(Vehicles.Models
(vehicleFactory: LowGradeVehicleFactory { }, expectedBikeType: typeof(Vehicles.Models

```

And two car tests

(`Vehicles.AbstractFactoryTest.Should_create_a_Car_of_the_specified_type`) executed with the following arguments:

```

(vehicleFactory: HighGradeVehicleFactory { }, expectedCarType: typeof(Vehicles.Models
(vehicleFactory: LowGradeVehicleFactory { }, expectedCarType: typeof(Vehicles.Models

```

If we review the tests' execution, both test methods are unaware of types. They use the Abstract Factory (`IVehicleFactory`) and test the `result` against the expected type without any knowledge of what they were testing, but the abstraction (or contract). That shows how loosely coupled the consumers (tests) and factories are.

In a real program, we would use the `ICar` or the `IBike` instances to execute some logic, compute statistics, or do anything relevant to that program. Maybe that could be a racing game or a rich person's garage management system, who knows!

The important part of this project is the abstraction of the object creation process. The consumer code was not aware of the implementations.

Note

The code of the second part of the project is part of another solution, named `MiddleEndVehicleFactory`, so you can compare the first version with its evolution.

To prove our design's flexibility, based on the Abstract Factory pattern, let's add a new concrete factory named `MiddleEndVehicleFactory`. That factory should return a `MiddleEndCar` or a `MiddleEndBike` instance. Once again, the car and bike are just empty classes (of course, in your programs they will do something):

```
public class MiddleGradeCar : ICar { }
public class MiddleGradeBike : IBike { }
```

The new `MiddleEndVehicleFactory` looks pretty much the same as the other two:

```
public class MiddleEndVehicleFactory : IVehicleFactory
{
    public IBike CreateBike() => new MiddleGradeBike();
    public ICar CreateCar() => new MiddleGradeCar();
}
```

As for the test class, we don't need to update the test methods (the consumers); we only need to update the setup to add new test data (see the highlighted lines):

```
public class AbstractFactoryTestCars : AbstractFactoryBaseTestData
{
    public AbstractFactoryTestCars()
    {
        AddTestData<LowGradeVehicleFactory, LowGradeCar>();
        AddTestData<HighGradeVehicleFactory, HighGradeCar>();
        AddTestData<MiddleEndVehicleFactory, MiddleGradeCar>();
    }
}
public class AbstractFactoryTestBikes : AbstractFactoryBaseTestData
{
    public AbstractFactoryTestBikes()
    {
        AddTestData<LowGradeVehicleFactory, LowGradeBike>();
        AddTestData<HighGradeVehicleFactory, HighGradeBike>();
        AddTestData<MiddleEndVehicleFactory, MiddleGradeBike>();
    }
}
```

If we run the tests, we now have six passing tests (two theories with three test cases each). So, without updating the consumer (the `AbstractFactoryTest` class), we were able to add a new family of vehicles, the middle-end cars and bikes; kudos to the Abstract Factory pattern for that wonderfulness!

Conclusion

Abstract Factory is an excellent pattern to abstract away the creation of object families, isolating each family and its concrete implementation, leaving the consumers unaware of (decoupled from) the family being created at runtime.

We talk more about factories in the next chapter; meanwhile, let's see how the Abstract Factory pattern can help us follow the **SOLID** principles:

- **S**: Each concrete factory has the sole responsibility of creating a family of objects. You could combine Abstract Factory with other creational patterns such as the **Prototype** and **Builder** patterns for more complex creational needs.
- **O**: The consumer is open to extension but closed for modification; as we did in the “expansion” sample, we can add new families without modifying the code that uses it.
- **L**: We are aiming at composition, so there's no need for any inheritance, implicitly discarding the need for the LSP. If you use abstract classes in your design, you need to make sure you don't break the LSP when creating new abstract factories.
- **I**: By extracting an abstraction that creates other objects, it makes that interface very focused on one task, which is in line with the ISP, creating flexibility at a minimal cost.
- **D**: By depending only on interfaces, the consumer is not aware of the concrete types that it is using.

Next, we explore the last design pattern of the chapter.

The Singleton design pattern

The Singleton design pattern allows creating and reusing a single instance of a class. We could use a static class to achieve almost the same goal, but not everything is doable using static classes. For example, implementing an interface or passing the instance as an argument cannot be done with a static class; you cannot pass static classes around, you can only use them directly.

We are exploring the Singleton pattern in this chapter because it relates to dependency injection. Knowing about the patterns in this order should help you with the next chapter.

In my opinion, the Singleton pattern in C# is an anti-pattern. Unless I cannot rely on dependency injection, I don't see how this pattern can serve a purpose. That said, it is a classic, so let's start by studying it, then move to a better alternative in the next chapter.

Here are a few reasons why we are covering this pattern:

- It translates into a singleton scope in the next chapter.
- Without knowing about it, you cannot locate it, nor try to remove it, nor avoid its usage.
- It is a simple pattern to explore, which is excellent for a first chapter about design patterns.
- It leads to other patterns, such as the **Ambient Context** pattern.

Goal

The Singleton pattern limits the number of instances of a class to one. Then, the idea is to reuse the same instance subsequently. A singleton encapsulates both the object logic itself and its creational logic. For example, the Singleton pattern could lower the cost of instantiating an object with a large memory footprint since it's instantiated only once.

Can you think of a SOLID principle that gets broken right there?

The Singleton pattern promotes that one object must have two responsibilities, breaking the **Single Responsibility Principle (SRP)**. A singleton is the object and its own factory.

Design

This design pattern is straightforward and is limited to a single class. Let's start with a class diagram:

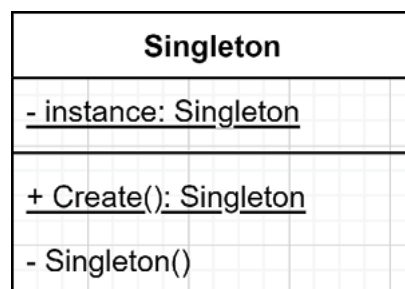


Figure 6.6: Singleton pattern class diagram

The `Singleton` class is composed of the following:

- A private static field that holds its unique instance.
- A public static `Create()` method that creates or returns the unique instance.
- A private constructor, so external code cannot instantiate it without passing by the `Create` method.

Note

You can name the `Create()` method anything or even get rid of it, as we see in the next example. We could name it `GetInstance()`, or it could be a static property named `Instance` or bear any other relevant name.

Now, in code, it can be translated to the following:

```
public class MySingleton
{
    private static MySingleton? _instance;
    private MySingleton() { }
    public static MySingleton Create()
    {
        if(_instance == default(MySingleton))
        {
            _instance = new MySingleton();
        }
        return _instance;
    }
}
```

```

    }
}

```

We can see in the following unit test that `MySingleton.Create()` always returns the same instance:

```

public class MySingletonTest
{
    [Fact]
    public void Create_should_always_return_the_same_instance()
    {
        var first = MySingleton.Create();
        var second = MySingleton.Create();
        Assert.Same(first, second);
    }
}

```

And voilà! We have a working Singleton pattern, which is extremely simple—probably the most simple design pattern that I can think of.

Here is what is happening under the hood:

1. The first time that a consumer calls `MySingleton.Create()`, it creates the first instance of `MySingleton`. Since the only constructor is private, it can only be created from the inside. You cannot instantiate `MySingleton` (using `new MySingleton()`) from the outside of the class because there is no public constructor.
2. That first instance is then persisted to the `_instance` field for future use.
3. When a consumer calls `MySingleton.Create()` a second time, it returns the `_instance` field, reusing the previous (and only) instance of the class.

If you want your singleton to be thread-safe, you may want to lock the instance creation, like this:

```

public class MySingletonWithLock
{
    private readonly static object _myLock = new();
    private static MySingletonWithLock? _instance;
    private MySingletonWithLock() { }
    public static MySingletonWithLock Create()
    {
        lock (_myLock)
        {
            if (_instance == default)
            {
                _instance = new MySingletonWithLock();
            }
        }
        return _instance;
    }
}

```

In the preceding code, we make sure two threads are not attempting to access the `Create` method simultaneously, to ensure that they are not getting different instances. We could use **double-checked locking** to optimize that pattern, but instead, we explore another, shorter way of achieving thread safety.

An alternate (better) way

Previously, we used the “long way” of implementing the Singleton pattern and had to implement a thread-safe mechanism. Now that classic is behind us. We can shorten that to get rid of the `Create()` method, like this:

```
public class MySimpleSingleton
{
    public static MySimpleSingleton Instance { get; } = new MySimpleSingleton();
    private MySimpleSingleton() { }
}
```

This way, you can use the singleton instance directly through its `Instance` property, like this:

```
MySimpleSingleton.Instance.SomeOperation();
```

We can prove the correctness of that claim by executing the following test method:

```
[Fact]
public void Create_should_always_return_the_same_instance()
{
    var first = MySimpleSingleton.Instance;
    var second = MySimpleSingleton.Instance;
    Assert.Same(first, second);
}
```

By doing this, our singleton becomes thread-safe as the property initializer creates the singleton instance instead of nesting it inside an `if` statement. It is usually best to delegate responsibilities to the language or the framework whenever possible.

Beware of the arrow operator

It may be tempting to use the arrow operator `=>` to initialize the `Instance` property like this: `public static MySimpleSingleton Instance => new MySimpleSingleton();`, but doing so would return a new instance every time. This would defeat the purpose of what we want to achieve. On the other hand, the property initializer is run only once.

The arrow operator makes the `Instance` property an expression-bodied member, which is the equivalent of creating the following getter: `get { return new MySimpleSingleton(); }`. Consult *Appendix A* for more information about expression-bodied statements.

The use of a static constructor would also be a valid, thread-safe alternative, once again delegating the job to the language.

Code smell – Ambient Context

That last implementation of the **Singleton** pattern led us to the **Ambient Context** pattern. We could even call the Ambient Context an anti-pattern, but let's just state that it is a consequential code smell.

I don't like ambient contexts for multiple reasons. First, I do my best to stay away from anything global. Globals can be very convenient at first because they are easy to use. They are always there and accessible whenever needed: easy. However, they can have many drawbacks in terms of flexibility and testability.

When using an ambient context, the following occurs:

- The system will most likely become **less flexible**. A global object is harder to replace and cannot easily be swapped for another object. Also, the implementation cannot be different based on its consumer.
- Global objects are hard to mock, which can lead to a system that is **hard to test**.
- The system can become **brittle**; for example, if some part of your system messes up your global object, that may have unexpected consequences on other parts of your system, and you may have a hard time finding out the root cause of those errors.
- Another thing that does not help is the lack of isolation since consumers are usually directly coupled with the ambient context. Not being able to isolate components from those global objects can be a hassle, as stated in the previous points.

Fun fact

Many years ago, before the JavaScript frameworks era, I ended up fixing a bug in a system where some function was overriding the value of `undefined` due to a subtle error. This is an excellent example of how global variables could impact your whole system and make it more brittle. The same is true for the Ambient Context and Singleton patterns in C#; globals can be dangerous and annoying.

Rest assured that, nowadays, browsers won't let developers update the value of `undefined`, but back then, it was possible.

Now that we've talked about globals, an ambient context is a global instance, usually available through a static property. The Ambient Context pattern is not purely evil, but it is a code smell that smells bad. There are a few examples in .NET Framework, such as `System.Threading.Thread.CurrentPrincipal` and `System.Threading.Thread.CurrentThread`, that are scoped to a thread instead of being purely global like most static members. An ambient context does not have to be a singleton, but that is what they are most of the time. Creating a non-global (scoped) ambient context is harder, requires more work, and is out of the scope of this book.

Is the Ambient Context pattern good or bad? I'd go with both! It is useful primarily because of its convenience and ease of use while it is usually global. Most of the time, it could and should be designed differently to reduce the drawbacks that globals bring.

There are many ways of implementing an ambient context; it can be more complicated than a simple singleton, and it can aim at another, more dynamic scope than a single global instance. However, to keep it brief and straightforward, we are focusing only on the singleton version of the ambient context, like this:

```
public class MyAmbientContext
{
    public static MyAmbientContext Current { get; } = new MyAmbientContext();
    private MyAmbientContext() { }
    public void WriteSomething(string something)
    {
        Console.WriteLine($"This is your something: {something}");
    }
}
```

That code is an exact copy of the `MySimpleSingleton` class, with a few subtle changes:

- Instance is named `Current`.
- The `WriteSomething` method is new but has nothing to do with the Ambient Context pattern itself; it is just to make the class do something.

If we take a look at the test method that follows, we can see that we use the ambient context by calling `MyAmbientContext.Current`, just like we did with the last singleton implementation:

```
[Fact]
public void Should_echo_the_inputted_text_to_the_console()
{
    // Arrange (make the console write to a StringBuilder
    // instead of the actual console)
    var expectedText = "This is your something: Hello World!" + Environment.NewLine;
    var sb = new StringBuilder();
    using (var writer = new StringWriter(sb))
    {
        Console.SetOut(writer);
        // Act
        MyAmbientContext.Current.WriteSomething("Hello World!");
    }
    // Assert
    var actualText = sb.ToString();
    Assert.Equal(expectedText, actualText);
}
```

The property could include a public setter (`public static MyAmbientContext Current { get; set; }`), and it could support more complex mechanics. As always, it is up to you and your specifications to build the right classes exposing the right behaviors.

To conclude this interlude: try to avoid ambient contexts and use instantiable classes instead. We see how to replace a singleton with a single instance of a class using dependency injection in the next chapter. That gives us a more flexible alternative to the Singleton pattern.

Conclusion

The Singleton pattern allows the creation of a single instance of a class for the whole lifetime of the program. It leverages a `private static` field and a `private` constructor to achieve its goal, exposing the instantiation through a `public static` method or property. We can use a field initializer, the `Create` method itself, a static constructor, or any other valid C# options to encapsulate the initialization logic.

Now let's see how the Singleton pattern can help us (not) follow the SOLID principles:

- **S:** The singleton violates this principle because it has two clear responsibilities:
 1. It has the responsibility for which it has been created (not illustrated here), like any other class.
 2. It has the responsibility of creating and managing itself (lifetime management).
- **O:** The Singleton pattern also violates this principle. It enforces a single static instance, locked in place by itself, which limits extensibility. The class must be modified to be updated, impossible to extend without changing the code.
- **L:** There is no inheritance directly involved, which is the only good point.
- **I:** There is no interface involved, which is a violation of this principle.
- **D:** The singleton class has a rock-solid hold on itself. It also suggests using its static property (or method) directly without using an abstraction, breaking the DIP with a sledgehammer.

As you can see, the Singleton pattern does violate all the SOLID principles but the LSP and should be used with caution. Having only a single instance of a class and always using that same instance is a legitimate concept. However, we see how to properly do this in the next chapter, leading me to the following advice: do not use the Singleton pattern, and if you see it used somewhere, try refactoring it out. Another good idea is to avoid the use of `static` members as much as possible as they create global elements that can make your system less flexible and more brittle. There are occasions where `static` members are worth using, but try keeping their number as low as possible. Ask yourself if that `static` member or class could be replaced with something else before coding one.

Some may argue that the Singleton design pattern is a legitimate way of doing things. However, in ASP.NET Core I cannot agree with them: we have a powerful mechanism to do it differently, called dependency injection. When using other technologies, maybe, but not with .NET.

Summary

In this chapter, we explored our first GoF design patterns. These patterns expose some of the essential basics of software engineering, not necessarily the patterns themselves, but the concepts behind them:

- The Strategy pattern is a behavioral pattern that we use to compose most of our future classes. It allows swapping behavior at runtime by composing an object with small pieces and coding against interfaces, following the SOLID principles.
- The Abstract Factory pattern brings the idea of abstracting away object creation, leading to a better separation of concerns. More specifi-

cally, it aims to abstract the creation of object families and follow the SOLID principles.

- Even if we defined it as an anti-pattern, the Singleton pattern brings the application-level objects to the table. It allows creating a single instance of an object that lives for the whole lifetime of a program. The pattern violates most SOLID principles.

We also peeked at the Ambient Context code smell, which is used to create an omnipresent entity accessible from everywhere. It is often implemented as a singleton and is a global object usually defined using the `static` modifier.

In the next chapter, we finally jump into dependency injection to see how it helps us compose complex yet maintainable systems. We also revisit the Strategy, the Factory, and the Singleton patterns to see how to use them in a dependency-injection-oriented context and how powerful they really are.

Questions

Let's take a look at a few practice questions:

1. Why is the Strategy pattern a behavioral pattern?
2. How could we define the goal of the creational patterns?
3. If I write the code `public MyType MyProp => new MyType();`, and I call the property twice (`var v1 = MyProp; var v2 = MyProp;`), are `v1` and `v2` the same instance or two different instances?
4. Is it true that the Abstract Factory pattern allows us to add new families of elements without modifying the existing consuming code?
5. Why is the Singleton pattern an anti-pattern?

[Support](#) [Sign Out](#)